PROJECT

LEE

Complete Project Description

Founder operating system · architecture · behavior · deployment

---

Prepared from the current repository state · 2026-08-21

Purpose

A detailed, source-grounded explanation of what Project LEE is, how its engines and ledgers connect, how it protects truth and authorization, how the Console, Android companion, Manual, API, and Windows desktop layer fit together, and what remains to be proven.

Prepared for Lamont Labs

This document describes implementation and current maturity; it does not claim that every planned boundary is fully complete.

Executive summary

Project LEE is a founder operating system designed to preserve continuity across a person·s projects,

LEE is built around a strict separation between what is observed, what is inferred, what is authorized, and what is ultimately executed. It stores source-backed facts separately from interpretations, records provenance for beliefs and conclusions, preserves immutable event history, surfaces uncertainty, and requires human/governance approval before consequential external actions.

Current product surfaces

- Lee Console · the primary web operating interface for today·s brief, projects, portfolio intelligence, knowledge, evidence, governance, connectors, health, backups, economics, and system state.
- Android companion · a capture-first, local-first mobile interface for quick text, photo, and voice capture, waiting loops, alerts, approvals, and uncertainty visibility.
- Project LEE Manual · the living technical and operational documentation surface containing the architecture map, Constitution, glossary, systems descriptions, and version history.
- API server · the PostgreSQL-backed service boundary containing the engines, ledgers, event infrastructure, schedulers, governance, backup/restore, and internal service integrations.
- Windows desktop layer · an Electron shell and release pipeline that turn LEE into a normal installed application with a tray process, local runtime supervision, an NSIS installer, and GitHub Release automation.

Design goals

- Continuity is the primary product: LEE should remember how the system got here, not only the latest value.
- Evidence is visible: every important claim should point to source records, events, provenance, or an explicit diagnostic.
- The system fails honestly: unavailable, degraded, cached, uncertain, measured, estimated, and unavailable states are distinct.
- Governance is separate from reasoning: a model can recommend, but it cannot authorize its own consequential action.
- Specialist services remain replaceable and external: CIL and CerbaSeal are integrated by signed contracts, not duplicated inside LEE.

Architecture at a glance

The layered model

- Layer 0 · Identity: the versioned Identity Profile defines what kind of operating partner LEE is, how it should interrupt, ask, escalate, observe, or remain silent, and what it must protect. Identity is consulted before the Constitution on every request.
- Layer 1 · Foundations: the Constitution, append-only Event Log, typed Domain Events catalog, core PostgreSQL schema, migrations, and Brain Versioning. Foundation failure moves LEE into a recovery mode rather than allowing unsafe continuation.
- Layer 2 · Knowledge: facts, interpretations, strategic anchors, decision heuristics, institutional knowledge, assumptions, provenance, the Intelligence Graph, the Digital Twin Timeline, ownership, and knowledge aging.
- Layer 3 · Retrieval: the Query Engine, Context Economy, temporal retrieval, ownership checks, and the local Semantic Index. Retrieval is centralized so ranking, policy, confidence, freshness, and telemetry remain consistent.
- Layer 4 · Intelligence: Intent, Understanding, Curiosity, Strategy, Reflection, Explanation, Confidence, Uncertainty, and Simulation engines. These interpret and explain; they do not directly perform external actions.
- Layer 5 · Coordination: Orchestration, Scheduler, Policy, Governance, Resource, State, Operating Modes, Engine Lifecycle, Recovery Modes, and Capability Registry.
- Layer 6 · Operational Context: World State, Operational Memory, Initiative, Operational Intelligence, Executive Loop, Operational Confidence, Executive Objectives, Capacity Awareness, Resource Allocation, Reviews, and bounded Self-Improvement.
- Layer 6b · Portfolio Intelligence: Project Momentum, Opportunity detection, portfolio summaries, dependency blast radius, and Execution Readiness.
- Layer 7 · Internal Capability Services: external CIL reasoning and CerbaSeal governance deployments, each with separate credentials, databases, signed HTTP contracts, and health state.

- Layer 8 · Provider Layer: typed provider-neutral interfaces and adapters for communication, documents, development, scheduling, storage, and project bootstrap.
- Layer 9 · Interfaces and Observability: Console, Android, Brief Engine, System Economics, backups, Self-Test, System Manifest, and the desktop packaging layer.

Architectural principles

- Constitution above everything; policies are configurable but cannot override constitutional provisions.
- Facts and Interpretations are permanently separate ledgers.
- The Query Engine is the universal access layer for knowledge retrieval.
- Intent is a first-class persisted object.
- Context competes for a finite budget; relevance and exclusions are audited.
- Domain Events are typed contracts, and the Event Log is append-only.
- Engine lifecycle, dependencies, health, and recovery are explicit.
- The Why Chain is always present for important conclusions.
- Brain Versioning preserves continuity across migrations and software changes.
- Self-improvement can adjust approved output behavior, but never identity, Constitution, facts, governance, permissions, credentials, or ownership.

Data, memory, and truth

PostgreSQL foundation

LEE uses PostgreSQL as its authoritative persistence layer. The database contains foundation objects and events, epistemic ledgers, graph and timeline structures, scheduler state, engine capability state, provider records, operational intelligence, economics, backups, restore verification, and Self-Test evidence. The desktop plan preserves PostgreSQL rather than replacing it for packaging convenience.

The Event Log

The Event Log is an immutable audit backbone. Typed domain events carry event type, version, timestamp, correlation information, and structured payload. A database trigger protects append-only behavior. Durable delivery tracks subscriber cursors, retries, idempotency, and dead letters. Projectors can rebuild narrow projections deterministically and produce checkpoints, receipts, conflicts, and dry-run results.

Knowledge ledgers

- Fact Ledger · source-backed assertions with confidence, observation and verification times, freshness, and provenance.
- Interpretation Ledger · conclusions derived from facts, visibly labeled as interpretations and kept separate from the Fact Ledger.
- Strategic Anchor Ledger · owner-declared priorities and constraints that remain stable and are checked against recommendations.
- Decision Heuristic Ledger · patterns inferred from observed decisions, not owner-declared preferences.
- Institutional Knowledge Ledger · earned knowledge requiring at least three independent supporting outcomes and no unresolved contradiction.
- Assumption Ledger · named premises with confidence, expiry, invalidation, and affected-conclusion visibility.

Provenance and ownership

LEE records creator, modifier, verifier, import, generation, and current-owner information. The Why Chain connects conclusions to supporting facts, interpretations, events, and source references. Bootstrap is evidence-first and reads project structure, not secret values.

Knowledge aging and uncertainty

Freshness is independent from truth. A record may remain valid while becoming stale for retrieval. Confidence describes evidence quality; uncertainty describes instability; Trust describes subsystem reliability; Operational Confidence describes the quality of the current operating picture. These signals are never collapsed into a single misleading number.

Request and reasoning flow

Universal request pipeline

- 1. Authenticate and establish the owner or registered service identity.
- 2. Consult the Identity Profile.
- 3. Consult the Constitution and applicable policy.
- 4. Classify and persist Intent.
- 5. Assemble a ranked Context Packet through Query Engine and Context Economy.
- 6. Route reasoning through the model boundary and CIL where appropriate.
- 7. Produce source-backed facts, interpretations, explanations, recommendations, or a governed-action request.
- 8. Record confidence, uncertainty, provenance, cost, and domain events.
- 9. Require confirmation and CerbaSeal authorization before consequential execution.

CIL reasoning boundary

CIL is a separate Cognitive Infrastructure Layer. It provides tiered reasoning reuse: T1 trigram reuse, T2 vector similarity, and T3 frontier escalation. Context is budgeted before routing. Requests include correlation and authentication metadata. CIL unavailability is visible and causes graceful degradation rather than a false healthy state.

Explanation Engine

Explanations remain interpretations, not facts. They include source IDs, provenance, a Why Chain, audience calibration, and feedback. The system can explain not only a recommendation but why it believes the recommendation is justified and where confidence was lost.

Simulation

Simulation is read-only. It records assumptions, scenarios, outcomes, and matching structures; a simulation never becomes an external action merely because its projected result looks favorable.

Governance and safety

CerbaSeal boundary

CerbaSeal is an independent Governance Service. It returns ALLOW, HOLD, or REJECT with reason codes, evidence, policy version, decision envelope, and audit references. LEE never accesses its database directly.

Fail-closed execution

- Unknown actions do not execute.
- Unavailable governance produces HOLD or REJECT, never ALLOW.
- Consequential provider writes require owner confirmation.
- The writer receives a unique, unexpired CerbaSeal ALLOW immediately before mutation.
- Expired, replayed, malformed, or contradictory authorization is rejected.
- A HOLD remains visible as a pending governed action rather than silently disappearing.

Constitutional boundaries

- Provenance is non-negotiable.
- Internal routes are not exposed as public routes.
- Semantic embeddings remain local.
- There are no silent failures.
- The Event Log is append-only.
- Facts and interpretations never mix.
- Provider-specific payloads stay inside adapters.
- Bootstrap never reads secrets.
- CerbaSeal is fail-closed.
- Credentials never enter logs, models, backups, or contract documents.

Operational intelligence and the Executive Loop

Operational Intelligence
Operational Intelligence is a synthesis layer. It ranks what deserves attention from initiatives, operational memory, world state, current objectives, portfolio signals, capacity, and canonical records. It is not itself the canonical source of truth and does not automatically execute actions.

Executive Loop
The Executive Loop is a persisted operational heartbeat around Operational Intelligence. Its phases are Observe, Understand, Prioritize, Decide, Prepare, Wait, Review, and Repeat. It is interrupted by significant failures or governed holds, survives restart through persisted state, and continues in degraded mode while surfacing what is unavailable.

Proactive engines
- Initiative Engine · surfaces operational drift and emerging situations.
- Opportunity Engine · finds cross-project reuse, leverage, and strategic alignment.
- Operational Review · produces structured retrospectives on improvement, regression, failed assumptions, and value.
- Operational Memory · learns from how LEE is used and which recommendations are accepted.
- Resource Allocation · advisory attention allocation across the portfolio.
- Execution Readiness · multidimensional readiness rather than a single project status.

System Economics
System Economics replaces the older Cost Engine concept with a broader accounting contract. It distinguishes MEASURED, ESTIMATED, and UNAVAILABLE values across reasoning, storage, network, connector, and operational dimensions. Missing measurement is not silently converted to zero.

Connectors and external systems
Provider abstraction
External services are accessed through provider-neutral interfaces. Communication, document, development, scheduling, and storage adapters normalize provider-specific payloads into records and domain events before internal engines consume them.

Read-first synchronization
Connector syncs are normalized and auditable. External writes are a separate boundary and require explicit authorization. Provider-specific failures are visible in connector health and operational confidence.

CIL, CerbaSeal, and Replit AI Bridge
LEE contains contracts and adapters for these specialist systems, not duplicate implementations. CIL provides reusable reasoning. CerbaSeal governs consequential actions. The Replit AI Bridge can provide managed model access without forcing the desktop user to collect individual model-provider credentials.

Project Bootstrap
Bootstrap converts observable repository structure into first-draft project knowledge: architecture inventory, dependencies, documentation, and security observations. It is rerunnable, advisory, source-backed, and explicitly forbidden from reading secrets.

Interfaces
Lee Console
The Console is the owner-facing operational surface. It exposes the Today/Morning Brief experience, Ask Lee, projects, portfolio, people, objectives, timeline, evidence, facts, interpretations, assumptions, strategic anchors, institutional knowledge, opportunities, simulations, reviews, connectors, internal services, governance, System Economics, health, backups, Manifest, and settings.

Android companion
Android is capture optimized and local first. Captures persist locally before sync. Failed sync remains visible and retryable. The companion caches uncertainty and system-contract posture so offline operation does not imply false certainty. Pairing and partial sync behavior are explicit.

Project LEE Manual
The Manual is a living explanation of the architecture, Constitution, systems, vocabulary, interfaces, ownership boundaries, degradation behavior, and version history. It uses the same System Contract vocabulary as the API, Console, and Android.

System Manifest and System Contract
The Manifest describes live system identity, engines, health, schemas, dependencies, backup/recovery state, and provenance. The versioned System Contract makes the shared agreement executable: identity, runtime, availability, freshness, health, capabilities, events, permissions, risk, governance, human confirmation, economics, dependencies, and evidence maps.

Backup, restore, recovery, and continuity

Brain Versions
Brain snapshots represent versioned operational continuity. Canonical sorted-key JSON with date normalization is hashed with SHA-256 for integrity. Backups carry manifests, checksums, provenance targets, and replay lineage.

Restore safety
Restore verification runs in isolation before production writes. The system checks checksums, object counts, provenance, replayability, and compatibility. Invalid Time Machine references fail visibly. Legacy provenance references are repaired or reported as warnings rather than hidden.

Recovery Modes
Boot selection persists clean shutdowns, recovery agendas, boot history, and write restrictions. Foundation failure can enter Safe mode; knowledge degradation can enter ReadOnly; scheduler degradation can enter Manual; external connectivity degradation can enter Isolated.

Desktop continuity
The Windows desktop supervisor creates the LEE data directories, starts the private PostgreSQL runtime, runs migrations, verifies the Brain and Event Log, starts the API and Executive Loop, and preserves the private database URL across restarts. Closing the window minimizes to the tray; Exit LEE shuts down managed processes.

Windows desktop deployment

Installed experience
- Download Project-LEE-Setup-x64.exe from GitHub Releases.
- Double-click the installer; no Node, package manager, PowerShell, or manual database commands are required.
- The installer creates desktop and Start Menu shortcuts.
- LEE starts with a tray process and a normal desktop window.
- The local runtime supervisor manages PostgreSQL, migrations, API startup, health checks, and shutdown.
- CIL, CerbaSeal, Replit AI Bridge, and other specialist services remain external API-connected dependencies.

Release pipeline
Stable tags matching lee-v* trigger a Windows GitHub Actions build. The workflow installs dependencies, builds the shared API schemas, builds the API server, builds the Console, packages the Electron application with NSIS, computes SHA-256 checksums, runs disposable Windows installer validation, and only then publishes the release artifact.

Windows validation
The release validation covers clean installation, private database startup, migration failure reporting, the Exit LEE tray action, child-process cleanup, restart reuse of the same data/database directories, and the distinction between application software updates and Brain state.

Current boundary
The desktop packaging layer is implemented and the follow-on tasks have been merged. Code signing is still a release-environment concern; unsigned Windows builds may receive SmartScreen warnings until a certificate is configured.

API and persistence surface

Representative route groups
/api/healthz; /api/objects; /api/events; /api/sources; /api/facts; /api/interpretations; /api/query/*; /api/semantic/*; /api/explanations/*; /api/provenance/*; /api/governance/*; /api/internal/*; /api/bootstrap/*; /api/providers/*; /api/backups/*; /api/brain-versions/*; /api/reviews/*; /api/institutional/*; /api/self-improvement/*; /api/economics/*; /api/execution-readiness/*; /api/resource-allocation/*; /api/portfolio-dependency/*; /api/android/*; /api/recovery/*; /api/self-tests; /api/orchestration/*; /api/state/*; /api/operational-confidence; /api/system-manifest; /api/manifest; /api/contract.

Persistence domains
Foundation and identity; epistemic knowledge; experience and institutional learning; retrieval and query telemetry; graph and timeline projections; coordination and scheduler; operational context; portfolio intelligence; providers and internal service health; economics, backups, restore verification, and Self-Test.

Event producers and consumers
Producers include foundation, ledgers, Bootstrap, providers, governance, model routing, execution, scheduler, Executive Loop, Operational Intelligence, reviews, experience processing, self-improvement, economics, backups, and Self-Test. Consumers include durable delivery, operational memory, Executive Loop interruption, OIE refresh, Android push, timeline and graph projectors, and Self-Test evidence.

Validation and current maturity

Acceptance evidence
The Version 12 acceptance audit recorded 20 behavioral/runtime scripts, 48 passing tests, and successful live responses from health, Manifest, System Economics, Operational Confidence, backups, recovery, and Self-Test endpoints.

Audit interpretation
The audit classified 68 areas as PARTIAL and one as DIFFERENT rather than claiming full completion. This is intentional: the project has real persistence, runtime behavior, APIs, and focused proof, but many areas still require broader lifecycle, external-service, production-scale, or universal wiring evidence.

Known limitations
- CIL availability can be WARN/degraded when the configured endpoint is unavailable.
- Legacy backup provenance references can remain warnings while lineage repair proceeds.
- The Event Log is a strong immutable audit/history backbone, but not every canonical table is yet reconstructed exclusively from events.
- Some intuitive legacy route paths differ from the current canonical namespaced routes.
- Universal enforcement across every engine and every write path is not yet fully proven.
- Measured economics remain unavailable or estimated where the system lacks direct evidence.

What LEE is ultimately becoming
LEE is becoming a durable operating partner for a founder: one that understands the difference between memory and inference, between recommendation and authorization, between current state and historical truth, and between a missing signal and a zero value.

The long-term desktop experience is intentionally ordinary for the user and sophisticated underneath: install from GitHub Releases, open the LEE icon, and let the system manage its local runtime while preserving continuity. The complexity belongs in the supervisor, recovery model, evidence map, and release pipeline·not in a PowerShell prompt.

The project·s defining characteristic is not the number of engines. It is the set of boundaries connecting them: provenance, epistemic separation, local-first continuity, fail-closed governance, provider isolation, explicit lifecycle, durable event history, and honest degradation.

Reference documents
VERSION_12_ACCEPTANCE_AUDIT.md · current acceptance rubric, runtime evidence, classification matrix, proof maps, and limitations.

DESKTOP_PACKAGING.md · Windows installer, local PostgreSQL lifecycle, first-launch checks, release workflow, and smoke-test requirements.

artifacts/lee-manual/src/data/architecture.ts · canonical layered architecture and failure/security boundaries.

artifacts/lee-manual/src/data/glossary.ts · canonical system vocabulary.

artifacts/api-server/src/lib/system-contract.ts · executable shared contract vocabulary and validation.

artifacts/api-server/src/lib/system-manifest.ts · live system Manifest projection.